

and the damping ratio is given by

$$\zeta = \frac{\omega_n R_2 C}{2} \quad (19.44)$$

The lock range is

$$\Delta\omega_L = 4\pi\zeta\omega_n \quad (19.45)$$

while the lock time remains $2\pi/\omega_n$. The pull-in range, using the active-PI loop filter, is limited by the VCO oscillator frequency. Consider the following example.

Example 19.3

Repeat Ex. 19.2 using the active-PI loop filter. The SPICE model for an op-amp (a voltage-controlled voltage source) seen in Fig. 20.19 can be used for the op-amp.

Using Eq. (19.43) in (19.44) gives

$$\zeta = \frac{R_2 C}{2} \sqrt{\frac{K_{PD} K_{VCO}}{N R_1 C}}$$

If we set the damping factor again to 1 we can write

$$4 = \frac{157 \times 10^6 R_2^2 C}{2\pi R_1}$$

Setting C to 10 pF and R_2 to 25k, then R_1 is 39k. The simulation results are seen in Fig. 19.31. The output clock is aligned to the center of the data. It's important to make sure that the step size used in the SPICE transient simulation isn't too big. For example, if a 100 MHz clock is used with a step size of 1 ns, it is likely that the loop either won't lock or if it does lock, both the VCO control voltage and thus the output data will oscillate. A maximum step size for 100 MHz signals may be 100 ps. Also note the sparse data pattern will increase the lock time (thus the reason for the long simulation time in Fig. 19.31).

Finally, it is important to remember that the VCO tuning scheme in Fig. 19.25 is not practical unless the components can be set manually (either off-chip or using fuses or switches on chip). Also, something very important that we are not discussing (yet) is power supply noise. The current-starved VCO's output frequency is not very stable with changes in V_{DD} . ■

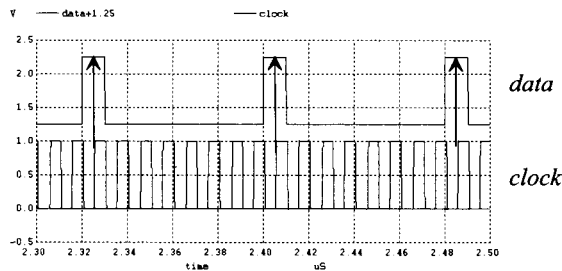


Figure 19.31 How the loop locks up on the center of the data.

19.3.2 PFD DPLL

Tri-State Output

A block diagram of a DPLL using the PFD with tri-state output is shown in Fig. 19.32. The phase transfer function is the same as Eq. (19.30)

$$H(s) = \frac{\phi_{clock}}{\phi_{data}} = \frac{K_{PDtri} K_F K_{VCO}}{s + \beta \cdot K_{PDtri} K_F K_{VCO}} \quad (19.46)$$

where

$$K_F = \frac{1 + sR_2C}{1 + s(R_1 + R_2)C} \quad (19.47)$$

When this filter is driven with the tri-state output, no current flows in R_1 or R_2 with the output in the high impedance state. The voltage across the capacitor remains unchanged. We can think of the filter, tri-state output as an ideal integrator with a transfer function

$$K'_F = \frac{1 + sR_2C}{s(R_1 + R_2)C} \quad (19.48)$$

Substituting this equation into Eq. 19.46 and rearranging results in

$$H(s) = \frac{K_{PDtri} K_{VCO} \frac{1 + sR_2C}{(R_1 + R_2)C}}{s^2 + s \frac{K_{PDtri} K_{VCO} R_2 C}{N(R_1 + R_2)C} + \frac{K_{PDtri} K_{VCO}}{N(R_1 + R_2)C}} = \frac{\phi_{clock}}{\phi_{data}} = \frac{f_{clock}}{f_{data}} \quad (19.49)$$

From this equation, the natural frequency is

$$\omega_n = \sqrt{\frac{K_{PDtri} K_{VCO}}{N(R_1 + R_2)C}} \quad (19.50)$$

and the damping factor is determined by solving

$$2\zeta\omega_n = \frac{K_{PDtri} K_{VCO} R_2 C}{N(R_1 + R_2)C} \quad (19.51)$$

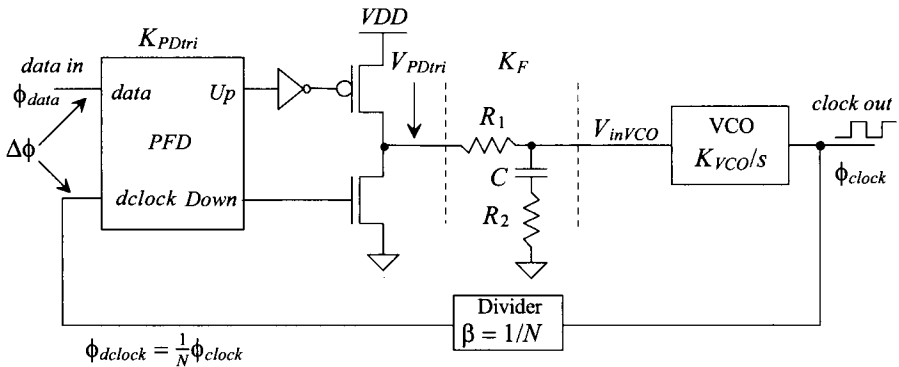


Figure 19.32 Block diagram of a DPLL using a sequential phase detector (PFD).

which results in a damping factor given by

$$\zeta = \frac{\omega_n}{2} \cdot R_2 C \quad (19.52)$$

The lock range is given by

$$\Delta\omega_L = 4\pi\zeta\omega_n \quad (19.53)$$

while the lock time, T_L , remains $2\pi/\omega_n$. The pull-in range is limited by the VCO operating frequency. The pull-in time is given by

$$T_P = 2R_1 C \cdot \ln \frac{(K_{VCO}/N) \cdot (VDD/2)}{(K_{VCO}/N)(VDD/2) - \Delta\omega} \quad (19.54)$$

where $\Delta\omega$ is the magnitude of the *input* frequency step.

Implementing the PFD in CMOS

The PFD seen in Fig. 19.10 can be implemented using inverters and nand gates as seen in Fig. 19.33. Simulating this circuit gives the results seen in Fig. 19.34. When the *dclock* is lagging, the *data* the output of the PFD is the *up* pulse (indicating that the edge of *dclock* needs to speed up or occur earlier in time, that is, the VCO's control voltage should increase). When *data* is lagging *dclock* the *down* pulse goes high indicating *dclock* should slow down.

We have some fine points that need to be discussed here. For example, what happens to *up* and *down* as the rising edges of *dclock* and *data* move close together? What we may get is some small glitches in the *up* and *down* signals. Or we may see both *up* and *down* staying low as the two pulses move closer together. In the PFD of Fig. 19.33, the delay through the two inverters can be used to set how *up* and *down* behave as

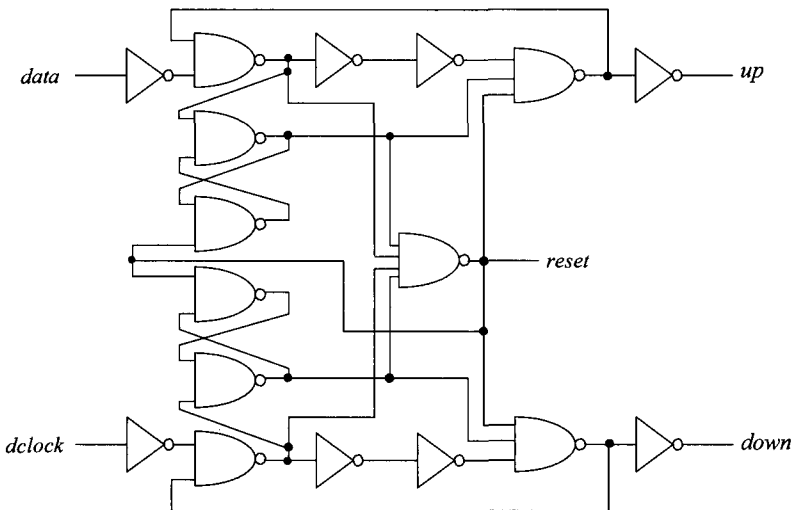


Figure 19.33 CMOS implementation of a PFD.

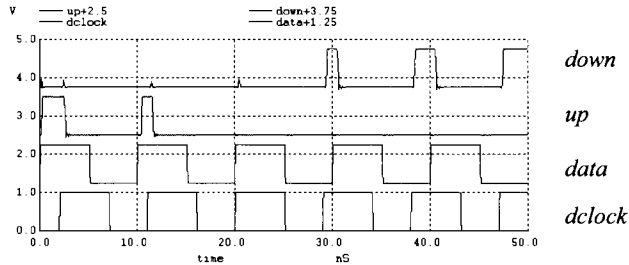


Figure 19.34 Simulation results for the PFD in Fig. 19.33.

the PFD's inputs move close together. If, for example, both stay low when the PFD's inputs get within 100 ps of each other, we get a *static phase error*. The loop won't lock any tighter than within 100 ps of its ideal position. Another way of looking at this is that as the phase difference, $\Delta\phi$, moves towards zero (see Fig. 19.11), the gain (the slope of the curve in Fig. 19.11) decreases. The phase detector is then said to have a *dead zone* where the gain of the phase detector (the slope of the curves) decreases.

Example 19.4

Design a DPLL using the tri-state topology seen in Fig. 19.32 that generates a clock signal at a frequency of 100 MHz from a 50 MHz square wave input. This application of the DPLL is called *frequency synthesis*.

The feedback path contains a divide by 2 circuit ($N = 2$). Let's use the VCO with the characteristics seen in Fig. 19.18

$$K_{VCO} = 1.57 \times 10^9 \text{ radians/V} \cdot \text{s}$$

The gain of the phase detector (knowing V_{DD} is 1 V) is

$$K_{PDtri} = \frac{1}{4\pi}$$

The lock range, Δf_L will be set to 20 MHz. Again let's set $\zeta = 1$. Using Eq. (19.53)

$$\Delta\omega_L = 4\pi\zeta\omega_n \rightarrow \omega_n = 10 \times 10^6 \text{ radians/V} \cdot \text{s}$$

Using Eq. (19.52) gives

$$R_2 C = 200 \text{ ns}$$

Let's set the capacitor to 10 pF and R_2 to 20k. Solving Eq. (19.50) for R_1 gives 42.5k ($= R_1$).

The simulation results are seen in Fig. 19.35. We should first notice that the VCO's control voltage doesn't have the excessive ripple like we had in the DPLL using the XOR gate phase detector. The response of the loop shows a nice $\zeta = 1$ shape. Again note that a DPLL loop's pull-in range is limited by the VCO's operating frequency (which, in this example uses the VCO from Fig. 19.18, which ranges from 50 to 150 MHz). A good exercise to perform at this point is to change the divider in the feedback path (to divide by 1, 2, 4, etc.) and the input frequency (a signal we've called *data*) and look at the robustness of the loop. ■

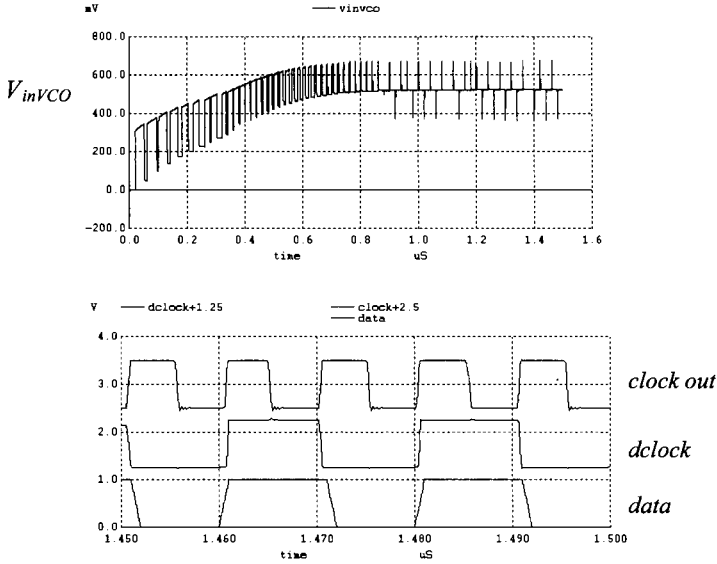


Figure 19.35 Simulation results for Ex. 19.4.

PFD with a Charge Pump Output

The PFD with a charge-pump output is seen in Fig. 19.36. A CMOS implementation of a DPLL using this configuration is, in general, preferred over the tri-state output because of the better immunity to power supply variations. In the tristate configuration seen in Fig. 19.32 note that when either the NMOS or PMOS switches are on, either V_{DD} or ground are connected directly to the loop filter. Power supply or ground noise can thus feed directly into the loop filter and then into the VCO's control voltage.

The loop filter integrates the charge supplied by the charge pump. The capacitor C_2 prevents $I_{pump} \cdot R$ from causing voltage jumps on the input of the VCO and thus frequency jumps in the DPLL output. In general, C_2 is set at about one-tenth (or less) of C_1 . The loop-filter transfer function neglecting C_2 is given by

$$K_F = \frac{1 + sRC_1}{sC_1} \quad (19.55)$$

The feedback loop transfer function is given by

$$H(s) = \frac{\phi_{clock}}{\phi_{data}} = \frac{K_{PDI}K_{VCO}(1 + sRC_1)}{s^2 + s\left(\frac{K_{PDI}K_{VCO}R}{N}\right) + \frac{K_{PDI}K_{VCO}}{NC_1}} \quad (19.56)$$

From the transfer function the natural frequency is given by

$$\omega_n = \sqrt{\frac{K_{PDI}K_{VCO}}{NC_1}} \quad (19.57)$$

and the damping factor is

$$\zeta = \frac{\omega_n}{2} \cdot RC_1 \quad (19.58)$$

The lock range and lock time remain the same (using the different values for the natural frequency and damping ratio) as the PFD with the tri-state output. Again, the pull-in range is set by the VCO oscillator frequency range. The pull-in time is given by

$$T_P = 2RC_1 \ln \left[\frac{(K_{VCO}/N) \cdot (I_{pump})}{(K_{VCO}/N) \cdot (I_{pump}) - \Delta\omega} \right] \quad (19.59)$$

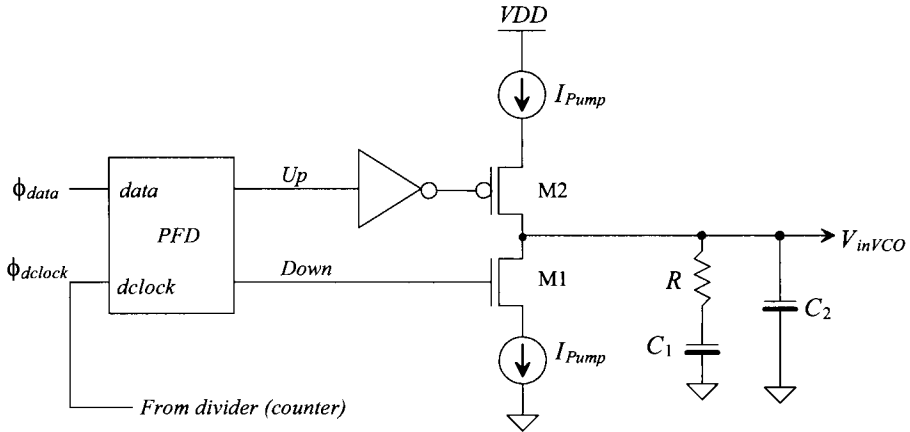


Figure 19.36 PFD using the charge pump.

Practical Implementation of the Charge Pump

The circuit seen in Fig. 19.36 is useful for illustrating the concept of the PFD with charge pump. However, in a practical circuit the fact that the sources of M1 and M2 charge to ground and V_{DD} respectively when M1 and M2 are off creates some design issues. For example, suppose that the source of M1 is discharged to ground when the *down* signal goes high. When M1 turns on, it doesn't supply the charge set by I_{pump} to the loop filter but rather, until the voltage across the current sink increases, behaves like a switch simply connecting the filter's input to ground (meaning that we are not controlling the signal that is applied to the loop filter). To get around this problem, consider the circuit seen in Fig. 19.37. The bias voltages come from diode-connected MOSFETs (to form current mirrors, see Ch. 20). When *up* and *down* are low, M1L and M2L are on. The pump-up current source, MPup, is driving the pump-down current source, MNdwn. When either *up* or *down* goes high, the output is connected to one of the current sources. The MOSFETs M1L,R and M2L,R simply steer the current to the loop filter. The only other concern we have with this topology is the fact that when M1L and M2L are both on, the voltage on their drains won't precisely match the voltage across the loop filter (V_{inVCO}). In other words, the voltage across each of the current sources (MPup and MNdwn) won't be the same as it is when they are connected to the output node. The result is that charge sharing between the parasitic capacitance on the drains of MPup and MNdwn and the capacitance used in the loop filter cause a static phase error or jitter. To eliminate this problem, an amplifier (see dashed lines in the figure) can be inserted to set the drain voltage of M1L and M2L to the same value V_{inVCO} .

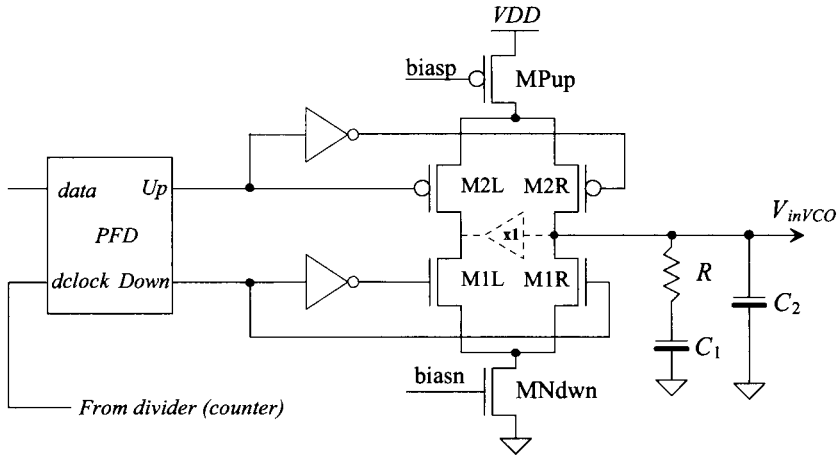


Figure 19.37 Practical implementation of the charge pump.

Example 19.5

Repeat Ex. 19.4 using the PFD and charge pump seen in Fig. 19.37.

One of the other benefits of using the charge pump is the fact that we can select I_{pump} , that is,

$$K_{PDI} = \frac{I_{pump}}{2\pi}$$

Again, let's set the lock range to 20 MHz. Using Eq. (19.53) with $\zeta = 1$ gives, again, $\omega_n = 10 \times 10^6$ radians/V · s. From Eq. (19.58) we get

$$RC_1 = 200 \text{ ns} \text{ so let's use } R = 20\text{k}, C_1 = 10 \text{ pF}, \text{ and } C_2 = 1 \text{ pF}$$

remembering that we generally set C_2 to one-tenth of C_1 . Using Eq. (19.57), we can now find the value of I_{pump}

$$10 \times 10^6 = \sqrt{\frac{I_{pump} \cdot 1.57 \times 10^9}{2\pi \cdot 2 \cdot 10 \times 10^{-12}}} \rightarrow I_{pump} = 8 \mu\text{A}$$

Because this value isn't that critical, we'll round it up to 10 μA . Figure 19.38 shows the simulation results. Notice the $\zeta = 1$ (or maybe a little less because the voltage does overshoot its final value by a little bit) shape of the VCO's control voltage. Let's modify the loop filter to show what V_{inVCO} would look like if $\zeta = 0.1$. All we have to do, from Eq. (19.58), is drop R by a factor of 10. The result is seen in Fig. 19.39. The control voltage is oscillating and the loop isn't locking. Of course, we can also increase the damping factor. The loop now behaves sluggishly and does not respond to fast changes in the input signal. For general design, where the process and temperature vary, it's better to center the design on a larger damping factor to avoid instability. ■

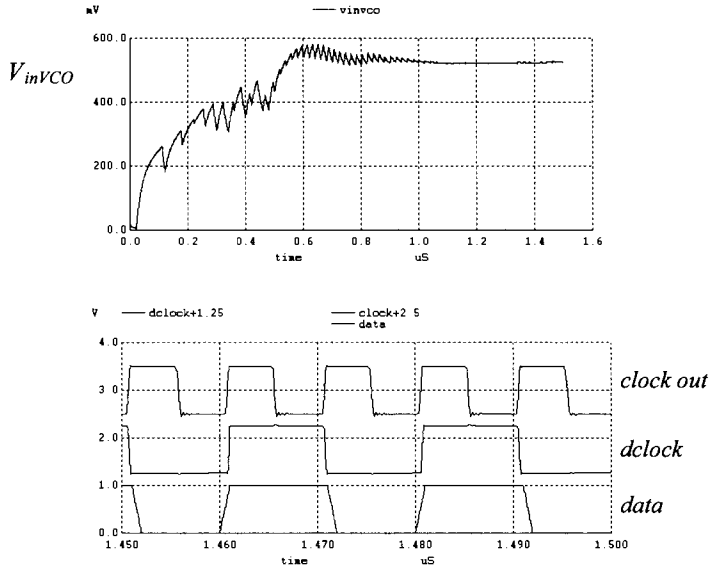


Figure 19.38 Simulation results for Ex. 19.5.

Discussion

When selecting values for the loop filter, we assumed that the output resistance of the phase detector was small (for the XOR and tri-state PD) compared to the impedances used in the loop filter. We also assumed that the input resistance of the VCO was infinite and the input capacitance of the VCO was small compared to the capacitance used in the loop filter. Considering the parasitics present in the DPLL is an important part of the design.

The center frequency of the VCO is critical for good DPLL performance when using the XOR gate with RC loop filter. If the center frequency, f_{center} , of the VCO (i.e., $V_{inVCO} = VDD/2$) does not match twice the input data rate, the DPLL will lock up at a phase different from $\pi/2$ (the input frequencies to any phase detector must be equal). The need for a precision center frequency is eliminated by using the XOR PD with an active-PI loop filter or by using the PFD. The other big benefit of using the PFD is that the VCO's gain can be larger.

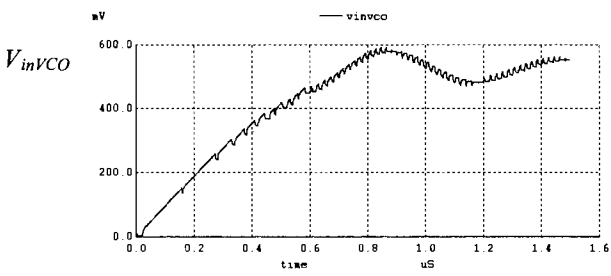


Figure 19.39 Showing what happens when the damping factor is reduced to 0.1.

Finally, selecting of the loop's damping factor, ζ , is very important. If the value of ζ is too small, the loop will have trouble locking or the output will jitter excessively when the loop is locked. This last problem is sometimes (gratuitously) called *jitter peaking* since a step in the DPLL's input frequency causes excessive overshoot in V_{inVCO} with small ζ .

19.4 System Considerations

System concerns are often the driving force behind the design of a DPLL. Referring to Fig. 19.1, we observe that the data transmitted through the channel should ideally arrive at the receiver with the same shape with which it was transmitted. In reality, the data becomes distorted. Distortion arises from nonlinearities in the receiver input amplifier and the finite bandwidth of the channel. To understand the conditions for distortionless transmission, consider the block diagram shown in Fig. 19.40. The system has a transfer function in the frequency domain of $H(f)$ and in the time domain $h(t)$. For distortionless transmission, we can relate the input and output of the system by

$$y(t) = K \cdot x(t - t_o) \quad (19.60)$$

where t_o is the time delay through the system and K is a constant. This equation shows that for distortionless transmission through a system the output is simply a scaled, time-delayed version of the input. An interesting observation can be made by taking the Fourier Transform of both sides of this equation,

$$Y(f) = K \cdot X(f)e^{-j2\pi ft_o} \quad (19.61)$$

The transfer function of a distortionless system can then be written as

$$H(f) = \frac{Y(f)}{X(f)} = Ke^{-j2\pi ft_o} \quad (19.62)$$

Figure 19.41 shows the magnitude and phase responses of a distortionless system. A system is distortionless when its amplitude response, $|H(f)|$, is a constant, K , and its phase response, $\angle H(f)$, is linear with a slope of $-2\pi t_o$ over all frequencies of interest.

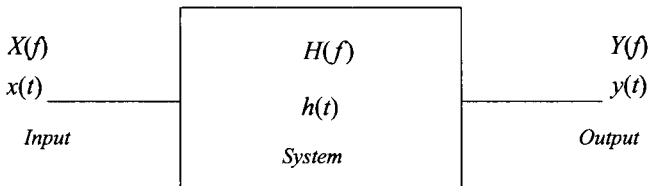


Figure 19.40 Representation of a system with input and output.

The responses shown in Fig. 19.41 are ideal. In practice, the magnitude response of a system may look similar to Fig. 19.42a. At higher frequencies, the magnitude rolls off. To compensate for this roll-off, or other imperfections, a circuit called an equalizer is added in series with the system (Fig. 19.42b). The equalizer has a transfer function in which its magnitude response increases with increasing frequency beyond a point (Fig. 19.42c). If the low-frequency gain of the equalizer is A/K and the low-frequency gain of the system is K , then the resulting gain of the system/equalizer combination is A .

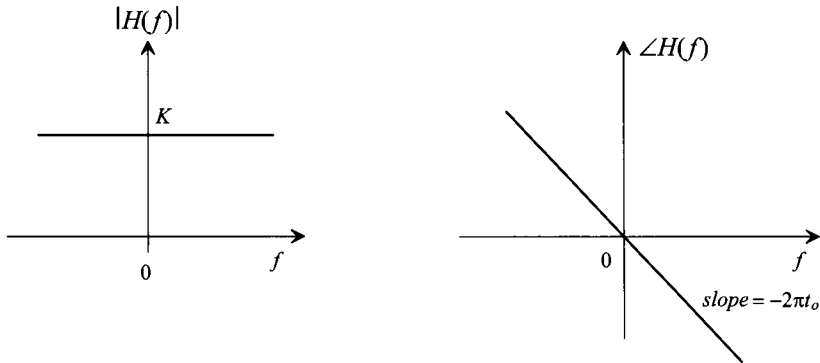


Figure 19.41 Magnitude and phase response of a distortionless system.

Another source of potential distortion occurs when the receiver input data is regenerated into digital levels. This was discussed back in Sec. 18.3. Timing errors occur when the input data is not precisely sliced through its middle (see Fig. 18.12). What makes slicing the data correctly even more difficult is the fact that the amplitude response of the channel can change with time and the data pattern can affect the average level of the data. There are two solutions to this problem. The first uses a circuit (see Fig. 18.29) that determines the peak positive and negative input analog amplitudes, averages the values, and feeds back the result to the comparator in the decision-making circuit. The second method encodes the digital data so that the duty cycle of the resulting encoded data is 50%. The encoding increases the channel bandwidth for a constant data rate.

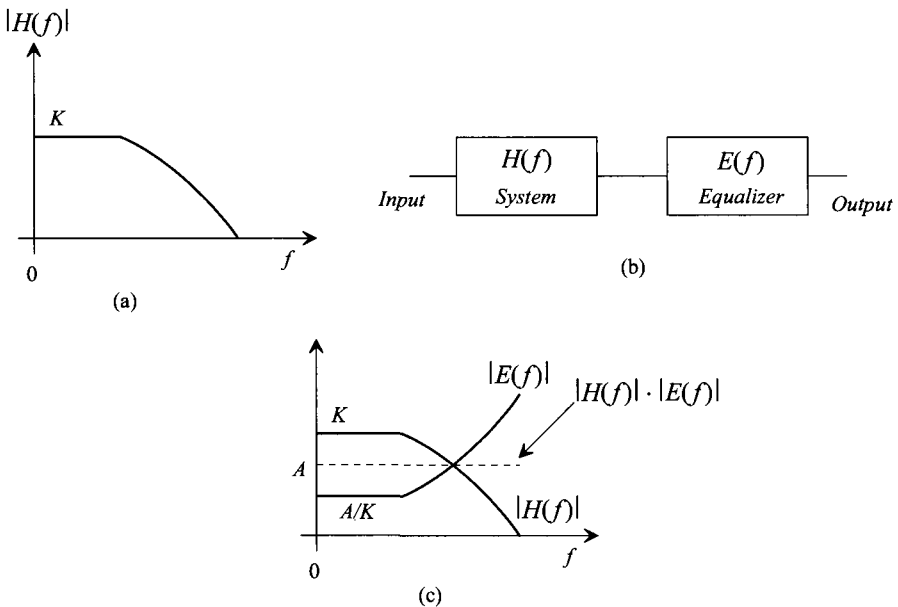


Figure 19.42 Using an equalizer to lower distortion in a system.

Encoding the data can eliminate the need for a decision circuit. If the resulting encoded data has a 50% duty cycle, it can be passed through a capacitor in the receiver, resulting in an analog signal centered around ground. The noninverting input of the comparator will then be connected to ground. The comparator will then be able to slice the data at the correct moments in time (in the middle of the data bit). An example of an encoding scheme is shown in Fig. 19.43. Encoding occurs in the transmitter prior to transmission over the channel. This particular encoding scheme is referred to as the bi-phase format, or more precisely, the bi-phase-level (sometimes called bi-phase-L or Manchester NRZ) format. The cost of using this scheme over the NRZ data format is increased channel bandwidth. Other encoding schemes are shown in Fig. 19.44.

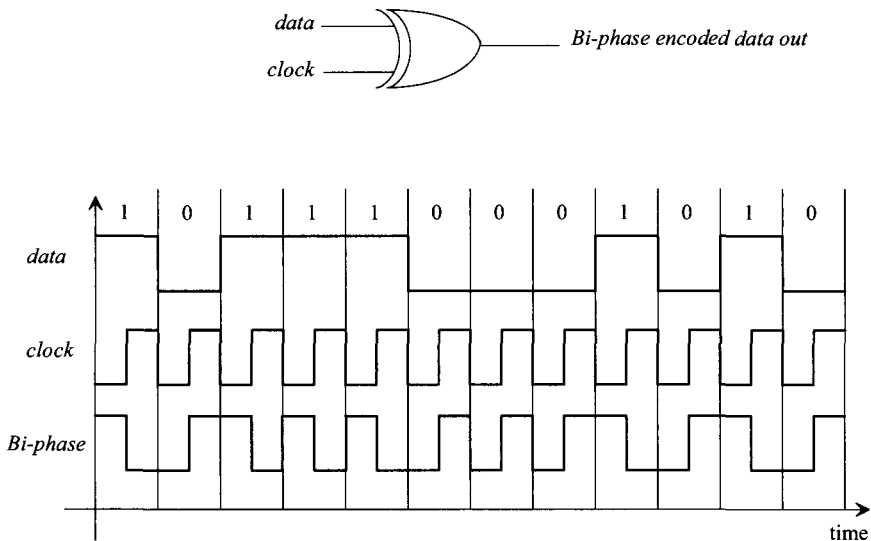


Figure 19.43 Bi-phase data encoding.

19.4.1 Clock Recovery from NRZ Data

One of the most important steps in the design of a communication system is the selection of the transmission format, that is, NRZ, bi-phase, or some other format, together with use of parity, cyclic redundancy code, or some other encoding format. In this section we discuss some of the considerations that go into the design of a clock-recovery DPLL in a system that uses NRZ.

Let's begin this discussion by considering the NRZ *data* and *clock* shown in Fig. 19.45. Let's further assume that these signals are the inputs to an XOR PD in a DPLL, which is not in lock since the clock is not aligned properly to the data. The resulting output of the XOR PD is shown in this figure as well. If we were to average this output using a loop filter, we would get $V_{DD}/2$. In fact, it is easy to show that shifting the *clock* signal in time has no effect on the average output of the PD. Why? To answer this question, let's use some numbers. Assume that a bit width of *data* is 10 ns (which is also the period of the *clock*). The frequency of the square wave resulting from the alternating